

MyLife

Patient-Owned Medical Records and Family Care Platform

Group 27 - Team Members

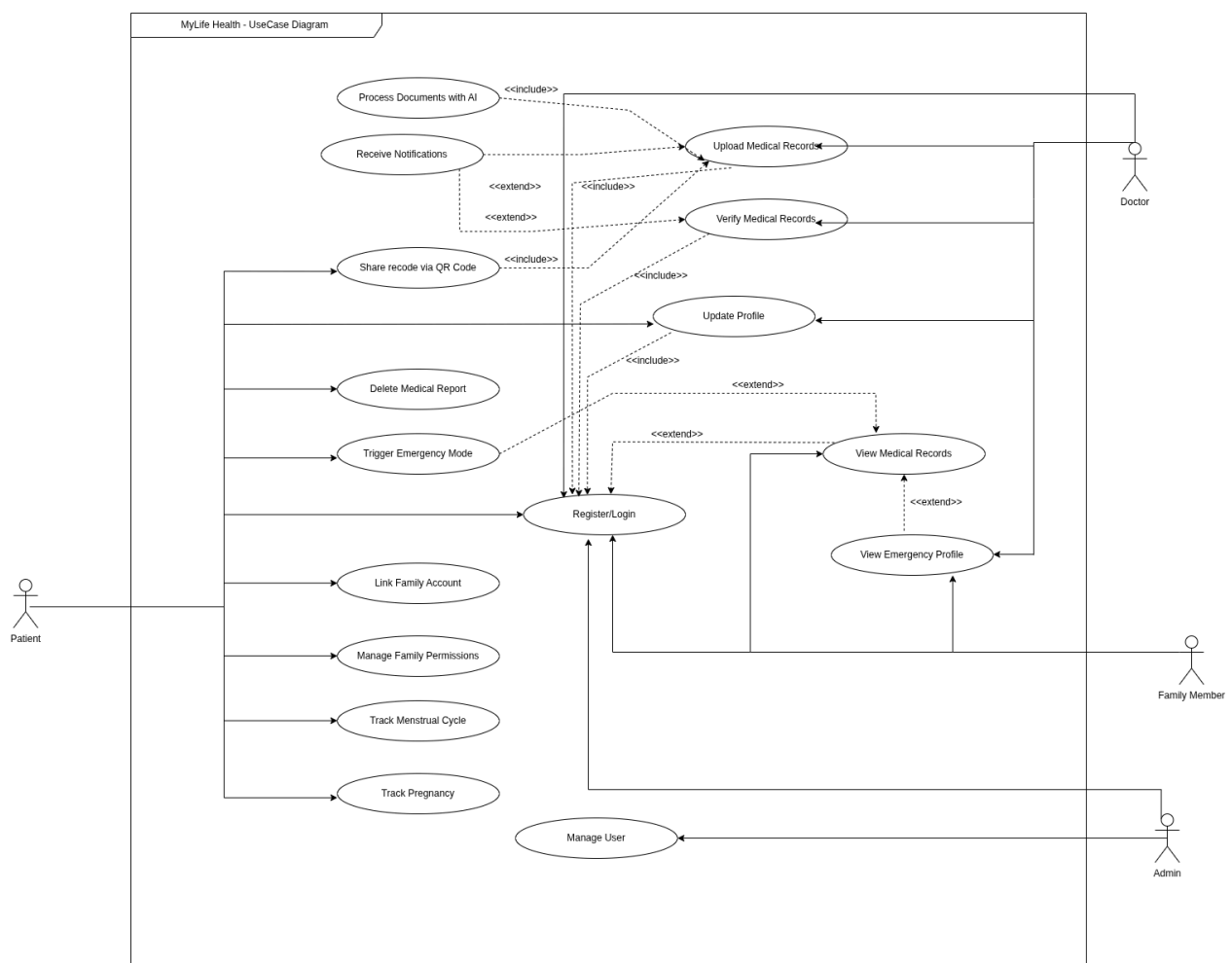
EC/2022/012 - T.M.D.P.M De Alwis

EC/2022/037 - K.G.H.D Bandara

EC/2022/016 - B.T. Bhanuka

EC/2022/055 - A.T.I Karunathilake

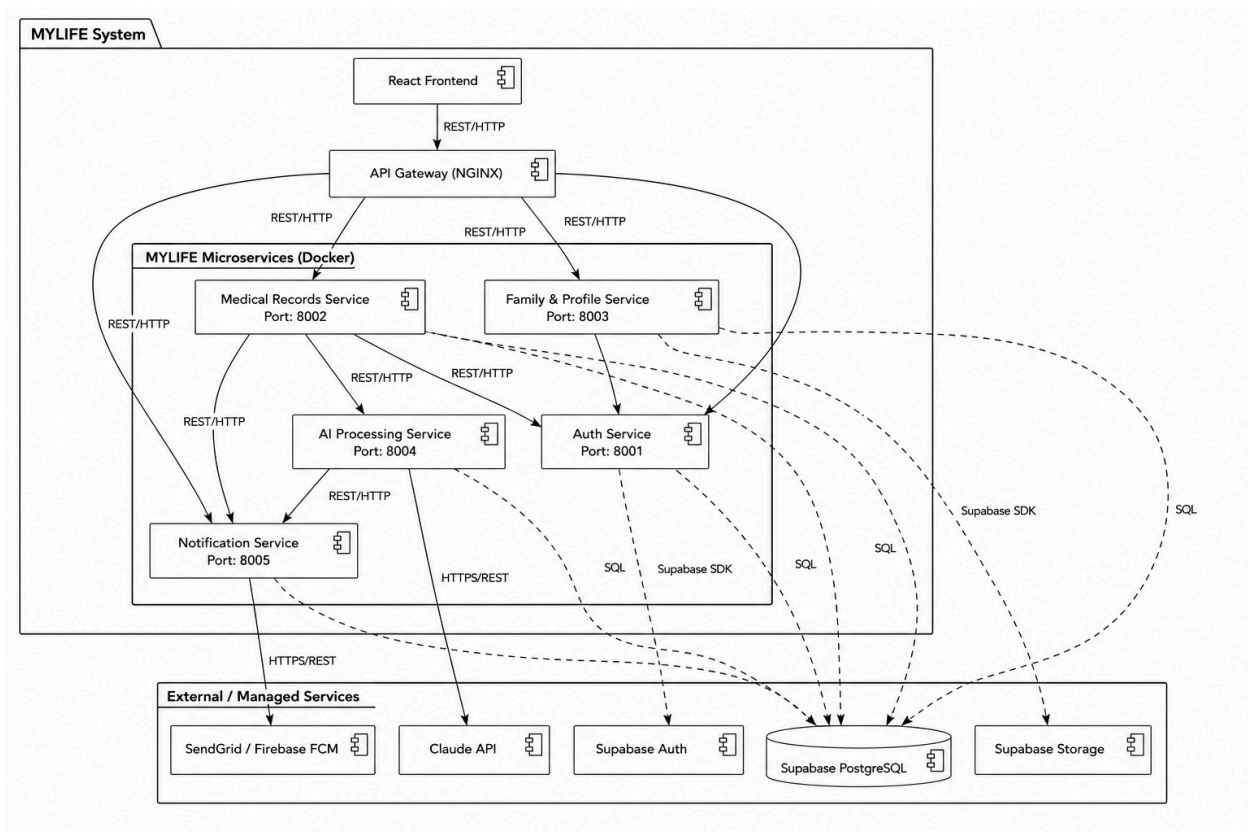
1) UseCase Diagram



- Patients utilize the app to register accounts, upload their medical documents, and generate QR codes to share specific records.
- The system automatically initiates AI document processing as a direct consequence of a patient uploading a new medical record.
- Verified doctors log into the platform to review and verify the medical records that patients have uploaded to the system.

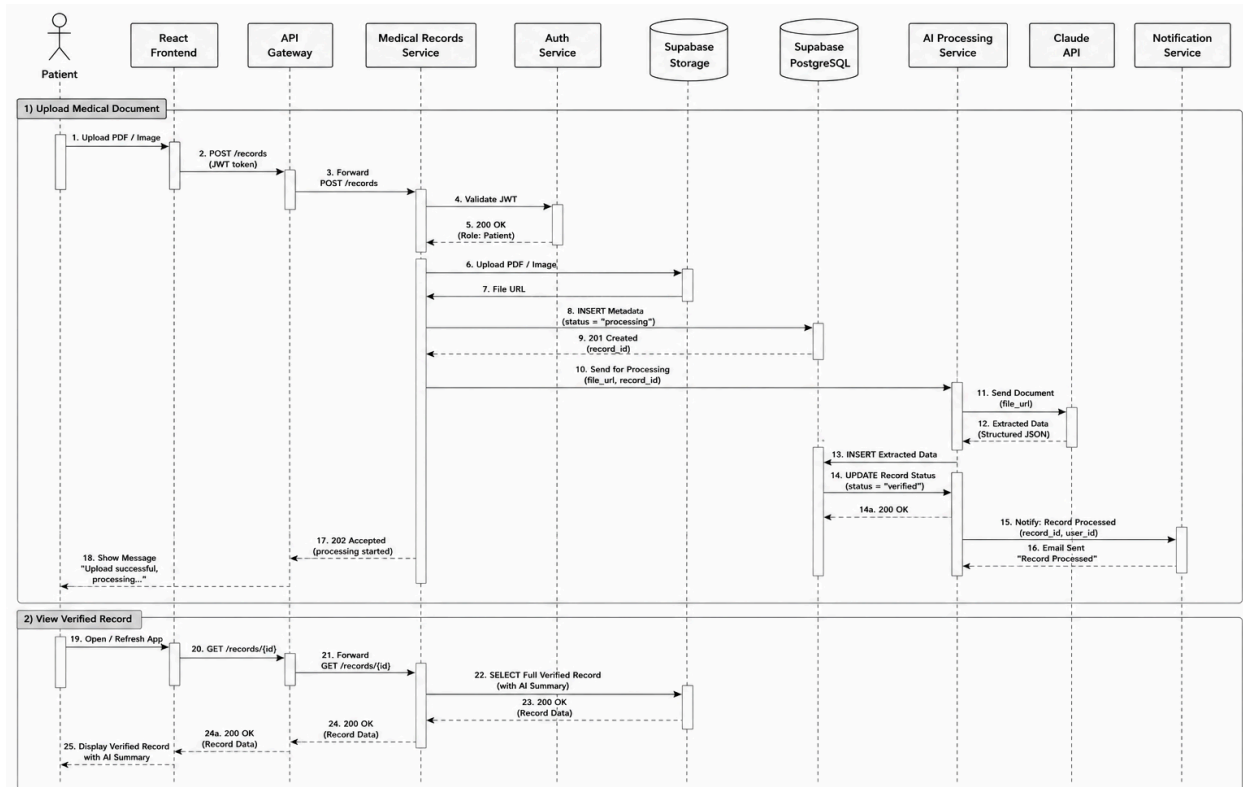
- Patients use the platform's family modules to track menstrual cycles, log pregnancy details, and manage family member access permissions.
- System administrators operate in the background to manage user accounts, oversee doctor verification, and maintain platform security.

2) Component Diagram



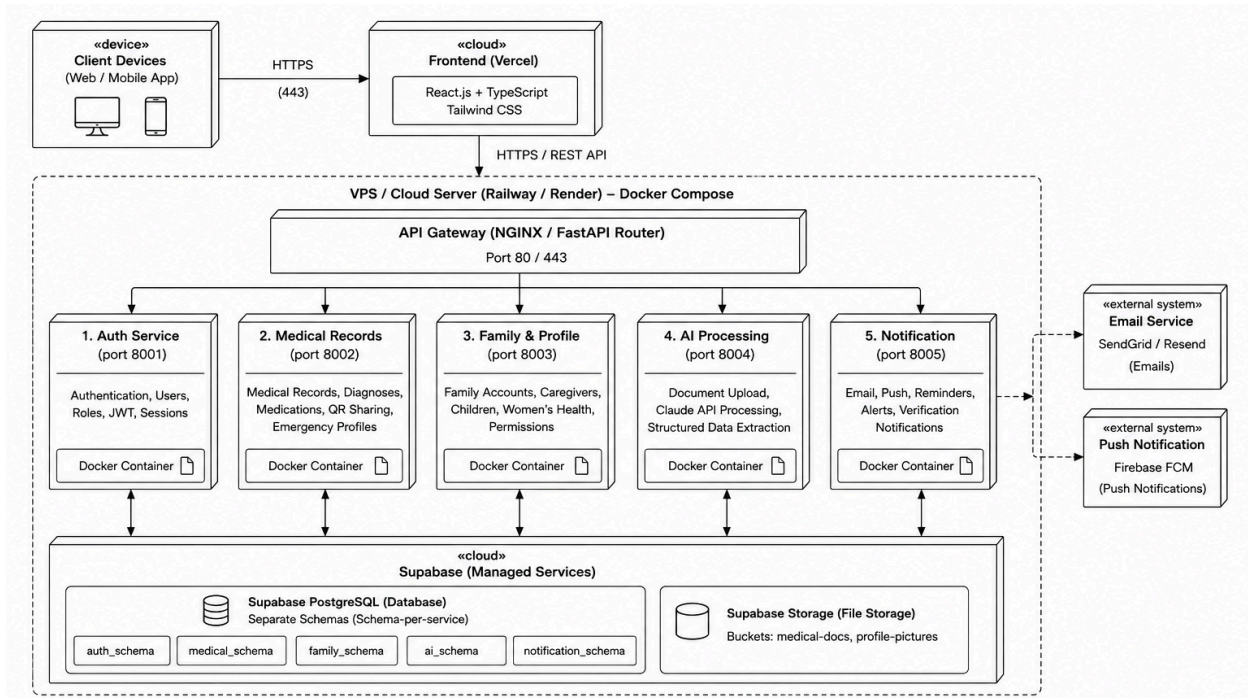
- The React frontend initiates REST HTTP requests which are all intercepted by the single NGINX API Gateway.
- The gateway evaluates the request path and routes traffic to the appropriate internal microservice (Auth, Medical, Family, AI, or Notification).
- The Medical Records Service securely communicates with the Auth service to validate JWTs before saving files to Supabase Storage.
- The AI Processing Service executes external HTTPS requests to the Claude API to perform data extraction on medical documents.
- All five core microservices independently read and write data to their own isolated schemas within the Supabase PostgreSQL database.

3) Sequence Diagram



- A patient uploads a file through the frontend, which is routed through the gateway to the Medical Records Service.
- The Medical Service validates the user's token, uploads the file to storage, and logs an initial "processing" state in the database.
- The AI Processing Service is triggered, sending the document to the Claude API with a prompt to extract structured medical data.
- Upon receiving the extracted data, the AI service saves it to the database, updates the record's status to "verified", and triggers an alert.
- The Notification Service emails the patient, who then opens the app to fetch and view the newly verified record alongside the AI summary.

4) Deployment Diagram



- The Vercel CDN continuously serves the static React frontend application directly to the user's web browser or mobile device.
- The Railway cloud server hosts the Docker Compose environment, physically running the NGINX gateway and the five Python/FastAPI containers.
- GitHub Actions continuously executes CI/CD pipelines, automatically deploying new code pushes to both Vercel and the Railway servers.
- The Dockerized microservices execute secure HTTPS requests out to the Supabase Cloud to manage authentication, relational data, and file storage.
- The backend services push outgoing data payloads to external cloud systems, interacting with the Anthropic cloud for AI and SendGrid/Firebase for alerts.